Computer Networks

CSA0712

CO4_Assessment Tool_2

Industry Problem Solving Task

Presented by: C. Sravanthi

192525463

SECTION B: TLS 1.3 Cryptography & Security

Question 4: Handshake Simplification & Speed

4.1 The 1-RTT Handshake

TLS 1.3 completes a secure handshake in 1 Round-Trip Time (1-RTT) by allowing the client to send important information in its first ClientHello message. The client includes a key share, which contains its contribution to the Diffie-Hellman key exchange.

The client makes the assumption that the server supports one of the cryptographic groups/key-exchange methods it offers. If the server accepts the offered key share, both sides can immediately calculate the shared secret.

The basic process is:

ClientHello + Key Share → ServerHello + Key Share → Encrypted communication

Thus, TLS 1.3 can establish encryption after only one complete round trip, making the connection faster than older TLS versions.

Key point: The client guesses/offers a suitable key-exchange group in the first Client Hello so that an additional round trip is not required.

4.2 Zero Round-Trip Time (0-RTT) Mode

In TLS 1.3, a returning client can use 0-RTT (Zero Round-Trip Time) when it has previously connected to the server.

During the previous connection, the server provides the client with information that can be used for session resumption. The client stores a Pre-Shared Key (PSK) or information from which the PSK can be derived.

When the client connects again, it can send:

ClientHello + PSK information + encrypted early data

in its first packet, without waiting for the server to respond.

A Pre-Shared Key (PSK) is a secret key that is already known or established by both communicating parties. In TLS 1.3, a PSK can be obtained from a previous secure session and used to resume that session quickly.

Therefore, 0-RTT reduces connection delay and is useful for applications such as HTTP.

4.3 Security Trade-off

The main security disadvantage of TLS 1.3 0-RTT is that the early data does not provide Perfect Forward Secrecy (PFS) in the same way as fresh ephemeral key exchange.

If an attacker records the encrypted 0-RTT traffic today and later obtains the secret information needed to recover the PSK, the attacker may be able to decrypt the previously recorded 0-RTT data.

Therefore, 0-RTT provides very high speed but has a security trade-off. It should mainly be used for data where this risk is acceptable.

Question 5: The Replay Attack Threat

5.1 Anatomy of a Replay Attack

A **Replay Attack** occurs when an attacker captures a valid message and sends the same message again later.

For example, suppose a user connects to a website using TLS 1.3 0-RTT over public university Wi-Fi. The user sends an encrypted request as early data.

An attacker monitoring the network may record this encrypted 0-RTT packet. The attacker does not necessarily need to decrypt it. Instead, the attacker can later **retransmit the same packet** to the server.

If the server accepts the request again, it may process the same action twice.

The main reason this is possible is that 0-RTT allows application data to be sent before a fresh handshake has completely established new session keys. Therefore, servers must take special precautions when processing 0-RTT requests.

Flow:

User → 0-RTT Request → Server

Attacker records request

Attacker → Replays same request → Server

If the request performs an important action, the repeated request can cause unwanted results.

5.2 Safe vs. Unsafe Actions

A request such as:

GET /news

is generally considered safe because a GET request normally only retrieves information. Repeating it does not normally change the server's data.

However:

POST /api/buy-item

can be dangerous because it may create a purchase or perform another action that changes the server's state. Replaying the same request could potentially cause the action to happen more than once.

Web developers should protect APIs by:

1. Avoiding 0-RTT for sensitive state-changing operations.
2. Using **unique request IDs or idempotency keys** to detect duplicates.
3. Keeping track of previously processed requests.
4. Designing important operations to be **idempotent** where possible.
5. Requiring a normal TLS handshake before performing highly sensitive operations.

Thus, 0-RTT should be used carefully for requests that can safely be repeated.

Question 6: Cryptographic Cleaning & Protocol Downgrades

6.1 Deprecation of Weak Ciphers

TLS 1.3 removed several old and weak cryptographic mechanisms instead of keeping them as optional choices.

For example, **Static RSA key exchange** was removed. Modern TLS 1.3 uses ephemeral Diffie-Hellman-based key exchange, which provides **Perfect Forward Secrecy**.

Older and weak hashing algorithms such as **MD5** and other obsolete cryptographic options were also removed because they have known security weaknesses.

Removing these algorithms provides several benefits:

- Prevents the use of outdated and vulnerable cryptography.
- Reduces the number of possible attack methods.
- Prevents administrators from accidentally selecting weak algorithms.
- Makes TLS implementations simpler and safer.
- Ensures that TLS 1.3 connections use modern cryptographic techniques.

Therefore, TLS 1.3 follows a **cleaner and stronger cryptographic design** rather than maintaining compatibility with insecure legacy algorithms.

6.2 Preventing Downgrade Attacks

A **Downgrade Attack** occurs when an attacker tries to force a client and server to use an older and weaker version of TLS.

TLS 1.3 includes mechanisms that help detect such attacks. The server places a special **downgrade indication in the ServerHello random value** when it is intentionally negotiating an older TLS version. This information is protected by the authenticated handshake.

If an attacker tries to modify the handshake to make the connection appear to use an older protocol version, the handshake authentication can fail.

TLS 1.3 also uses **transcript authentication**, meaning important handshake messages are cryptographically verified. Therefore, an attacker cannot simply modify the negotiated version without being detected.

In simple terms:

Attacker: “Use TLS 1.0 instead.”

Client/Server: “The handshake information does not match the expected authenticated values.”

Result: The downgrade attempt is detected and the connection can be terminated.

Thus, TLS 1.3 helps ensure that attackers cannot silently force a connection to use an older, vulnerable TLS version.